Income_Inequality_in_Los_angeles_Building_Permit_Activity

```
# Core Python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import streamlit as st

from snowflake.snowpark.context import get_active_session
session = get_active_session()
```

```
session.sql("USE ROLE TRAINING_ROLE").collect()
session.sql("USE DATABASE LA_PERMIT_DATA").collect()
session.sql("USE SCHEMA PUBLIC").collect()
```

```
records = session.table("PERMIT_RECORDS").to_pandas()
records.head()
```

```
census_tracts = session.table("CENSUS_TRACTS").to_pandas()
census_tracts.head()
```

```
# Convert date columns
for col in ["ISSUE_DATE", "STATUS_DATE"]:
    records[col] = pd.to_datetime(records[col], errors="coerce")

# Clean valuation (string → numeric)
records["VALUATION"] = pd.to_numeric(
        records["VALUATION"].str.replace("$", "").str.replace(",",""),
        errors="coerce"
    )

# Create CT key for joining to census
ct_numeric = pd.to_numeric(records["CENSUS_TRACT"], errors="coerce")
records["CT"] = (ct_numeric * 100) + 6037000000
```

```
# Join Permits and Census Tracts
records_with_census = records.merge(census_tracts, left_on="CT", right_on="CENSUS_TRACT", how="inner")
records_with_census.head()
```

```
# Create High and Low Income Groups
records_with_census["MED_HH_INCOME"] = pd.to_numeric(
    records_with_census["MED_HH_INCOME"],
    errors="coerce"
)

# Drop rows without income
records_with_census = records_with_census.dropna(
    subset=["MED_HH_INCOME"]
)

# Median split: bottom 50% = Low, top 50% = High
median_cutoff = records_with_census["MED_HH_INCOME"].median()

records_with_census["INCOME_BRACKETS"] = np.where(
    records_with_census["MED_HH_INCOME"] >= median_cutoff,
    "high-income",
    "low-income"
)

records_with_census["INCOME_BRACKETS"].value_counts()
```

```
# Classify Improvement Types from AI_DESCRIPTION
def improvement_types(text):
    text = text.lower()

    # Energy efficiency
    if "solar" in text or "hvac" in text or "heat pump" in text:
        return "Energy efficiency upgrades"

    # Accessibility
    elif "ada" in text or "ramp" in text or "wheelchair" in text:
        return "Accessibility modifications"

    # Repairs / Maintenance
    elif "roof" in text or "repair" in text or "replace" in text:
        return "Repair / Maintenance"

    # Interior remodels
    elif "kitchen" in text or "bath" in text:
        return "Interior remodel"

    # ADU
    elif "adu" in text or "accessory dwelling" in text:
        return "ADU"

    else:
        return "Other"
```

```
records_with_census["IMPROVEMENT_TYPE"] = (
    records_with_census["AI_DESCRIPTION"].fillna("")
    .apply(improvement_types)
)

records_with_census["IMPROVEMENT_TYPE"].value_counts()
```

```
records_with_census = records_with_census.dropna(
    subset=["ISSUE_DATE", "INCOME_BRACKETS"]
)
```

```
# Graph 1. Total Permit Count by Income Brackets
permit_counts = (
    records_with_census
        .groupby("INCOME_BRACKETS")["PCIS_PERMIT_NUM"]
        .nunique()
        .reindex(["low-income", "high-income"])
)

plt.figure()
ax = permit_counts.plot(kind="bar")
plt.title("Total Permit Count by Income Brackets")
plt.xlabel("Income Brackets")
plt.ylabel("Number of permits")
plt.xticks(rotation=0)
for i, value in enumerate(permit_counts):
    ax.text(i, value, f"{value:,}", ha="center", va="bottom")
plt.tight_layout()
plt.show()
```

```
# Graph 2. Total Permit Valuation by Income Brackets
valuation_by_income = (
    records_with_census
        .groupby("INCOME_BRACKETS")["VALUATION"]
        .sum()
        .reindex(["low-income", "high-income"])
)

plt.figure()
ax=valuation_by_income.plot(kind="bar")
plt.title("Total Permit Valuation by Income Brackets")
plt.xlabel("Income Brackets")
plt.ylabel("Total valuation ($)")
plt.xticks(rotation=0)
for i, value in enumerate(valuation_by_income):
    ax.text(i, value, f"{value:,}", ha="center", va="bottom")
plt.tight_layout()
plt.show()
```

```
# Graph 3. Improvement Types by Income Brackets
improvement_counts = (
    records_with_census
        .groupby(["IMPROVEMENT_TYPE", "INCOME_BRACKETS"])["PCIS_PERMIT_NUM"]
        .count()
        .unstack(fill_value=0)
)

improvement_counts = improvement_counts[["low-income", "high-income"]]

plt.figure()
ax=improvement_counts.plot(kind="bar")
plt.title("Improvement Types by Income Brackets")
plt.xlabel("Improvement type")
plt.ylabel("Number of permits")
plt.xticks(rotation=45, ha="right")
for container in ax.containers:
    labels = [f"{int(bar.get_height()):,}" for bar in container]
    ax.bar_label(
        container,
        labels=labels,
        fontsize=5,
        padding=3
    )

plt.tight_layout()
plt.show()
```

```
# Graph 4. Share of Energy & Accessibility Permits by Income Brackets

# Filter only Energy & Accessibility permits
subgroup = records_with_census[
    records_with_census["IMPROVEMENT_TYPE"].isin([
        "Energy efficiency upgrades",
        "Accessibility modifications"
    ])
]

subgroup_counts = (
    subgroup
    .groupby(["INCOME_BRACKETS", "IMPROVEMENT_TYPE"])["PCIS_PERMIT_NUM"]
    .count()
    .unstack(fill_value=0)
)

# Column-by-column division
row_totals = subgroup_counts.sum(axis=1)
share = subgroup_counts.copy()
for col in share.columns:
    share[col] = share[col] / row_totals

plt.figure()
ax=share.plot(kind="bar")
plt.title("Share of Energy & Accessibility Permits by Income Brackets")
plt.xlabel("Income Brackets")
plt.ylabel("Share of permits")
plt.xticks(rotation=0)
for container in ax.containers:
    ax.bar_label(container, fmt="%.2f", label_type="edge")
plt.tight_layout()
plt.show()
```

```
# Graph 5. Permit Trends Over Time by Income Brackets

# Extract year
records_with_census["YEAR"] = records_with_census["ISSUE_DATE"].dt.year

yearly_trend = (
    records_with_census
        .groupby(["YEAR", "INCOME_BRACKETS"])["PCIS_PERMIT_NUM"]
        .count()
        .unstack(fill_value=0)[["low-income", "high-income"]]
)

plt.figure()
yearly_trend.plot()
plt.title("Permit Activity Over Time by Income Brackets")
plt.xlabel("Year")
plt.ylabel("Number of permits")
plt.tight_layout()
plt.show()
```

```
# Dashboard
st.title("LA Building Permit Inequality Dashboard")
st.write("Decision-focused view of permit activity across income brackets (low vs high)")

base_df = records_with_census.copy()

# Sidebar Filters
st.sidebar.header("Filters")

income_filter = st.sidebar.multiselect(
    "Select Income Brackets:",
    options=sorted(base_df["INCOME_BRACKETS"].unique()),
    default=sorted(base_df["INCOME_BRACKETS"].unique())
)

df = base_df[base_df["INCOME_BRACKETS"].isin(income_filter)]

# Year column
df["YEAR"] = df["ISSUE_DATE"].dt.year

year_filter = st.sidebar.slider(
    "Select Year Range:",
    int(base_df["ISSUE_DATE"].dt.year.min()),
    int(base_df["ISSUE_DATE"].dt.year.max()),
    (
        int(base_df["ISSUE_DATE"].dt.year.min()),
        int(base_df["ISSUE_DATE"].dt.year.max())
    )
)

df = df[(df["YEAR"] >= year_filter[0]) & (df["YEAR"] <= year_filter[1])]

st.sidebar.write(f"Records: {len(df):,}")

st.subheader("Key Metrics")

total_permits = df["PCIS_PERMIT_NUM"].count()
low_count = df[df["INCOME_BRACKETS"] == "low-income"]["PCIS_PERMIT_NUM"].count()
high_count = df[df["INCOME_BRACKETS"] == "high-income"]["PCIS_PERMIT_NUM"].count()

col1, col2, col3 = st.columns(3)
col1.metric("Total Permits", f"{total_permits:,}")
col2.metric("Low-Income Permits", f"{low_count:,}")
col3.metric("High-Income Permits", f"{high_count:,}")

# GRAPH 1 — TOTAL PERMIT VALUATION (UPDATED)

st.subheader("Total Permit Valuation by Income Brackets")

g1_valuation = (
    df
    .groupby("INCOME_BRACKETS")["VALUATION"]
    .sum()
)

st.bar_chart(g1_valuation)

st.caption(
    "This chart shows total dollar valuation of permitted construction "
    "in low and high income census tracts."
)

# GRAPH 2 — PERMIT TRENDS OVER TIME
st.subheader("Permit Trends Over Time by Income Brackets")
```

```
g2 = (
    df
    .groupby(["YEAR", "INCOME_BRACKETS"])["PCIS_PERMIT_NUM"]
    .count()
    .unstack(fill_value=0)
)

st.line_chart(g2)

st.caption(
    "This chart shows how permit activity has evolved over time by income brackets."
)
```